

Voice2Baby

Technical Onboarding Guide

For the Capstone and VIP development teams

VCU Computer Science Department · VCU Health NICU

Dr. Tamer Nadeem

Live at voice2baby.com

Contents

1	Preface: how to use this guide	3
2	Introduction: what Voice2Baby is	3
3	Core concepts you'll keep seeing	3
3.1	The two halves of a web app	3
3.2	How the client and server talk	4
3.3	Data & security	4
3.4	Tooling you'll use	4
4	System architecture	5
5	Technology stack	5
6	Repository layout	6
7	Local development setup	7
7.1	Prerequisites	7
7.2	Clone and install	7
7.3	Environment variables	7
7.4	Start the local infrastructure	8
7.5	Run the apps	8
8	Backend deep dive	8
8.1	Application entry (<code>server/index.js</code>)	8
8.2	Authentication & authorization	8
8.3	Data access	9
8.4	Utilities (<code>server/utls/</code>)	9
8.5	Scheduler (<code>scheduler.js</code>)	9
8.6	IoT status subscriber (<code>iotSubscriber.js</code>)	9
8.7	Migrations (<code>migrate.js</code>)	9
9	API reference	9
9.1	Auth — <code>/auth</code>	9
9.2	Admin — <code>/admin</code> (admin only)	10
9.3	Babies — <code>/babies</code>	10
9.4	Rooms — <code>/rooms</code>	10
9.5	Recordings — <code>/recordings</code>	10
9.6	Devices — <code>/devices</code>	11
10	Database schema	11
11	Frontend deep dive	11
12	Infrastructure & AWS	12
12.1	Our server vs. managed services	12
12.2	Networking basics (so the rest makes sense)	12
12.3	EC2: the server itself	13

12.4	nginx: the web server and “reverse proxy”	13
12.5	HTTPS, TLS, and certificates (the padlock)	13
12.6	PM2 — keeping the Node app alive	14
12.7	The managed AWS services (what, why, what they need, when)	14
12.8	Credentials: which secret lives where	15
12.9	At a glance	16
13	The Raspberry Pi speaker	16
13.1	What runs on the Pi	16
13.2	Zero-touch provisioning: how a blank Pi becomes a known device	16
13.3	Configurable WiFi (<code>wifi.txt</code>)	17
13.4	The image: what’s inside it and how it’s modifiable	17
13.5	Certificates & identity: which files, and how they’re obtained	18
14	Deployment & operations	19
14.1	CI/CD	19
14.2	Manual operations on EC2	19
14.3	Domain & TLS	19

1 Preface: how to use this guide

This document is the single starting point for anyone joining the Voice2Baby project. It explains *what* the system is, *how* the pieces fit together, and *how* to set it up and work on it.

Read Sections 2–6 first for the big picture, then follow Section 7 to get the app running on your machine. Use the remaining sections as reference while you work.

2 Introduction: what Voice2Baby is

Babies in a Neonatal Intensive Care Unit (NICU) often cannot be with their parents. Research shows newborns benefit from hearing their parents' voices.

Voice2Baby lets a parent record their voice *remotely* from home; a nurse or clinician reviews it; and it is then played on a small speaker (a Raspberry Pi) at the baby's incubator — on a schedule or on demand.

The three human roles (Personas):

- **Parent:** records and submits voice messages for their baby.
- **Nurse:** reviews recordings, schedules or triggers playback, or rejects it with a message.
- **Admin:** manages users (nurses, parents), rooms, babies, and the physical speaker devices; has full oversight.

End-to-end flow:

1. Parent records audio in the web app; it uploads to cloud storage (S3).
2. The recording enters `pending_review`. A nurse reviews it.
3. On approval it is scheduled (or played immediately).
4. At play time the backend sends a command over MQTT to the assigned Raspberry Pi in the baby's room.
5. The Pi downloads the audio and plays it through the speaker, reporting status back over MQTT.

3 Core concepts you'll keep seeing

If some words below are new, read this once and the rest of the guide will make sense. Skip it if you already know these.

3.1 The two halves of a web app

Client / frontend

- The part that runs *in the user's browser*; the buttons and screens.
- Ours is written in **React**. It holds no secrets and can be read by anyone, so it must never contain passwords or private keys.

Server / backend

- A program running on a computer *we* control that the client talks to. It enforces the rules, holds the secrets, and reads/writes the database.

- Ours is `Node.js` + `Express`.

3.2 How the client and server talk

HTTP(S)

The language browsers and servers speak. **HTTPS** is HTTP with encryption.

API “Application Programming Interface” — the set of URLs the server exposes for the client to call. Ours lives under `/api/v1`.

REST / endpoint

A style of API where each *endpoint* (a URL like `/api/v1/babies`) represents a thing, and the HTTP *method* says what to do: `GET` (read), `POST` (create), `PATCH` (update), `DELETE` (remove).

JSON

The text format requests and responses are written in (ex. `{"name": "Sara"}`).

Port A numbered “door” on a computer for a specific program. Our API listens on port 3000; the web server on 80 (HTTP) and 443 (HTTPS); the dev client on 5173.

Status code

A number the server returns: `2xx`=success, `4xx`=you did something wrong (e.g. 401 unauthorized, 404 not found), `5xx`=the server broke.

3.3 Data & security

Database / SQL

Where structured data is stored in tables. We use MySQL and talk to it in SQL (`SELECT`, `INSERT`, ...).

Migration

A small, ordered SQL script that changes the database shape (e.g. add a column), so every environment’s schema stays in sync. Ours live in `database/migrations/`.

Authentication vs. authorization

Authentication = proving who you are (login). *Authorization* = what you’re allowed to do (a parent can’t open the admin screens). We do the first with a JWT, the second with roles.

JWT “JSON Web Token” — a signed string the server gives you at login. The browser sends it back on every request to prove it’s you; it’s signed with a secret, so it can’t be forged.

Hashing (bcrypt)

Passwords are never stored as-is; we store a one-way *hash*. You can check a password against the hash but can’t reverse it.

Environment variables / `.env`

Settings and secrets (database password, API keys) kept *out* of the code, in a file the program reads at startup. Never commit real `.env` files.

3.4 Tooling you’ll use

Node.js & npm

Node runs our JavaScript server. `npm` installs the libraries it needs: `package.json` lists

them; `npm install` downloads them into `node_modules` (never committed).

Git / branch / PR

Git tracks code history. You make a *branch*, commit changes, and open a *pull request* to merge into `main`.

CI/CD

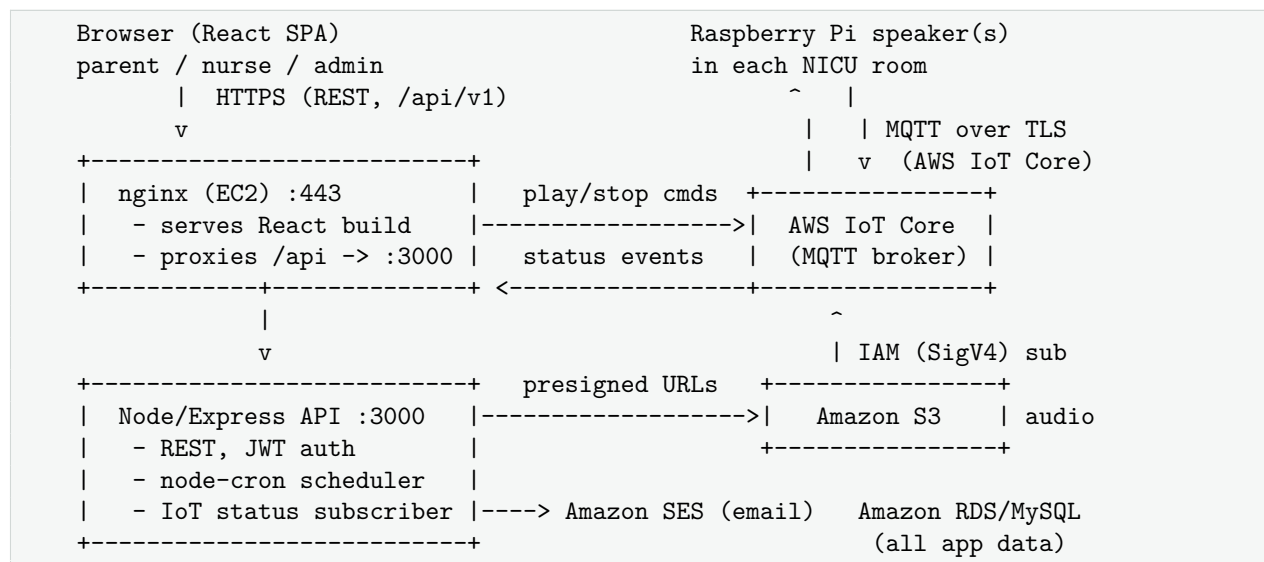
“Continuous Integration / Deployment” — automation that ships code when you push. Ours (GitHub Actions) deploys `main` to the server automatically.

Docker / container

A way to run a program inside a small, throwaway Linux environment so it behaves the same on any machine. We use it for the local database and to build the Pi image on a Mac.

4 System architecture

Voice2Baby has four moving parts: a **web client**, a **Node API server**, **AWS cloud services**, and the **Raspberry Pi speakers**.



Listing 1: High-level architecture

Why these pieces:

- The browser talks only to the API over HTTPS. In production the site and the API are served from the *same* domain (nginx serves the static React build at / and proxies /api to Node), so browser requests are same-origin.
- Audio files never pass through the database — they live in S3; the DB stores metadata and S3 keys. The Pi fetches audio via a short-lived **presigned URL**.
- Play/stop commands and device status flow over **MQTT** through AWS IoT Core, not HTTP — devices are intermittently connected and behind hospital firewalls, which MQTT handles well.

5 Technology stack

Layer	Technology	Role
Frontend	React 19 + Vite, React Router 7, Axios, lucide-react icons	Single-page app for all three roles.
Backend	Node.js (v20) + Express 5	REST API at /api/v1.
Auth	JSON Web Tokens (jsonwebtoken), bcryptjs	Stateless login; role-based access.
Database	MySQL 8 (Amazon RDS in prod), mysql2 driver	All application data.
Audio storage	Amazon S3 (@aws-sdk/client-s3 + presigner)	Stores recordings; presigned URLs for upload/download.
Messaging	AWS IoT Core (MQTT), aws-iot-device-sdk-v2	Play/stop commands + device status.
Email	Amazon SES (@aws-sdk/client-ses)	Invites, password resets, notifications.
Scheduling	node-cron (in-process)	Fires due playbacks every minute.
Device	Raspberry Pi OS Lite, Python 3, ffmpeg (ffmpeg)	The physical speaker; MQTT client + audio playback.
Provisioning	AWS IoT Fleet Provisioning + Lambda	Zero-touch device registration from one golden image.
Infra	EC2 (Elastic IP), nginx, Certbot/Let's Encrypt, Route 53, PM2	Hosting, TLS, DNS, process management.
CI/CD	GitHub Actions → SSH → EC2	Auto-deploy on push to main.
Local AWS	LocalStack + Docker Compose	Emulates S3/SES for local dev.

6 Repository layout

```

Code/
  client/
    src/
      api/
      components/
      context/ hooks/
      pages/
      public/
    server/
      index.js
      config/db.js
      middleware/
      routes/
      utils/
      scheduler.js
      iotSubscriber.js
      React + Vite web app
      axios client (baseUrl from VITE_API_URL)
      layout, ui primitives, recordings widgets
      AuthContext, useAuth
      admin/ nurse/ parent/ + Login/Signup/Reset
      favicon.svg, logos/
      Node + Express API
      app entry: mounts routes, starts scheduler + IoT sub
      MySQL connection pool
      auth.js (JWT), requireRole.js (RBAC)
      auth, admin, babies, rooms, recordings, devices
      s3, ses, iot, tokens, ids, recordingCascade
      node-cron: triggers due playbacks
      subscribes to devices/+status over MQTT

```

migrate.js	applies database/migrations/*.sql on startup
database/	
schema.sql	full current schema (fresh installs)
seed.sql	sample/seed data
migrations/	001_..., 002_... ordered, idempotent
pi/	Raspberry Pi speaker
pi_subscriber.py	MQTT client + ffmpeg playback
provision_agent.py	first-boot self-registration
apply-wifi.sh	configures WiFi from wifi.txt (home/enterprise/open)
firstboot/	firstboot.sh + nicu-firstboot.service
systemd/	pi-subscriber, pi-provision, nicu-wifi units
build-thin-image.sh	injects our bootstrap into official Raspberry Pi OS
build-image-docker.sh	builds the image on a Mac via Docker (no Pi needed)
scripts/aws/	one-time AWS IoT fleet-provisioning setup
docs/	this guide
docker-compose.yaml	local MySQL + LocalStack
*.md	deployment / device / domain runbooks

7 Local development setup

7.1 Prerequisites

- Node.js v20+ and npm
- Docker + Docker Compose (for local MySQL and LocalStack)
- Git

7.2 Clone and install

```
git clone https://github.com/mariamelwirish/voice-2-baby
cd Code
(cd server && npm install)
(cd client && npm install)
```

7.3 Environment variables

Copy `.env.example` to `server/.env` and fill it in. Key variables:

Variable	Meaning
PORT	API port (default 3000).
JWT_SECRET	/ Token signing secret + lifetime.
JWT_EXPIRES_IN	
DB_HOST/PORT/NAME/USER/ PASSWORD	MySQL connection.
AWS_REGION	AWS region (e.g. <code>us-east-1</code>).
AWS_S3_BUCKET	Bucket holding audio.
AWS_ENDPOINT_URL	(Local only) point AWS SDKs at LocalStack.
SES_SENDER_EMAIL	Verified “from” address for email.
IOT_ENDPOINT	/ AWS IoT Core endpoint.
AWS_IOT_ENDPOINT	

Variable	Meaning
DEVICE_PROVISION_SECRET	Shared secret for device self-provisioning.
FRONTEND_URL / API_URL	Used in email links.
ALLOWED_ORIGINS	Comma-separated CORS allow-list (dev: http://localhost:5173).

For the client, `client/.env.production` sets `VITE_API_URL`. In dev, Vite proxies `/api` to the backend (see `client/vite.config`).

7.4 Start the local infrastructure

```
docker compose up -d          # starts MySQL + LocalStack (see docker-compose.yaml)
# load schema into the local DB (first time):
mysql -h 127.0.0.1 -u <user> -p <db> < database/schema.sql
mysql -h 127.0.0.1 -u <user> -p <db> < database/seed.sql # optional sample data
```

`localstack-init/` bootstraps the S3 bucket / SES identity in LocalStack.

7.5 Run the apps

```
# terminal 1 - API (auto-runs pending migrations on start)
cd server && npm run dev          # http://localhost:3000

# terminal 2 - web client
cd client && npm run dev          # http://localhost:5173
```

Open <http://localhost:5173>. The client (:5173) calls the API (:3000) through Vite's proxy, so both are same-origin in the browser and CORS does not interfere.

8 Backend deep dive

8.1 Application entry (`server/index.js`)

Loads env, creates the Express app, applies CORS + JSON body parsing, mounts the six route modules under `/api/v1`, then: connects the MySQL pool, runs pending migrations (aborts start on failure unless `RUN_MIGRATIONS=false`), starts listening, starts the cron scheduler, and starts the IoT status subscriber.

8.2 Authentication & authorization

- `POST /api/v1/auth/login` verifies the password (bcrypt) and returns a signed JWT plus basic user info. The client stores the token and sends it as `Authorization: Bearer <token>` on every request.
- `middleware/auth.js` verifies the JWT and attaches `req.user`.
- `middleware/requireRole.js` guards routes by role, e.g. `requireRole('admin')`. Runs after `authenticateToken`.
- Accounts are created by invite: an admin creates a nurse/parent, SES emails a signup link with a one-time token; `POST /auth/signup` sets the password. `forgot-password/reset-password` follow the same token pattern (SHA-256 hashed token, expiry, single use).

8.3 Data access

`config/db.js` exposes a `mysql2` promise pool. Queries use parameter placeholders (?) — never string interpolation — to prevent SQL injection. The pool is single-statement; only `migrate.js` uses a multi-statement connection.

8.4 Utilities (`server/utls/`)

- `s3.js` — upload + **presigned URL** generation for audio.
- `ses.js` — all outbound email (invites, resets, notifications).
- `iot.js` — publishes play/stop commands to `devices/<code>/*.`
- `tokens.js` — generate/hash invite & reset tokens.
- `ids.js` — ID/code generation (e.g. `PI-####`).
- `recordingCascade.js` — cascade cleanup (DB rows + S3 objects) on delete.

8.5 Scheduler (`scheduler.js`)

An in-process `node-cron` job runs every minute, finds `schedules` that are due, and triggers the same playback path as the manual `POST /recordings/:id/play`. An `isProcessing` guard prevents overlapping runs (which would double-fire a play command).

8.6 IoT status subscriber (`iotSubscriber.js`)

Holds a persistent MQTT subscription to `devices/+status` using the backend's IAM credentials (SigV4 over WebSocket — not a device certificate). It records device online/offline (via MQTT connect + Last-Will) and playback lifecycle events (started/played/stopped/fetch_failed) back into the database.

8.7 Migrations (`migrate.js`)

Applies each `database/migrations/*.sql` that hasn't run yet, in filename order, recording applied files in `schema_migrations`. Runs automatically on server start and via `npm run migrate`. Write migrations idempotent (guard ALTERs with an `information_schema` check) and also update `schema.sql` for fresh installs.

9 API reference

All routes are under `/api/v1`. Unless noted, requests require a valid JWT; role restrictions are enforced by `requireRole`.

9.1 Auth — `/auth`

Method	Path	Purpose
POST	<code>/login</code>	Email + password → JWT.
POST	<code>/signup</code>	Activate an invited account (token + new password).
POST	<code>/forgot-password</code>	Email a reset link (generic response).
POST	<code>/reset-password</code>	Set a new password from a reset token.

9.2 Admin — /admin (admin only)

Method	Path	Purpose
POST	/nurses	Create + invite a nurse.
POST	/parents	Create + invite a parent (and link a baby).
POST	/resend-invite	Re-send an invite email.
GET	/nurses	List nurses.
GET	/nurses/:id/activity	A nurse's activity log.
GET	/parents	List parents.
DELETE	/nurses/:id	Permanently delete a nurse (cascade).
DELETE	/parents/:id	Permanently delete a parent (cascade).
DELETE	/parents/:id/babies/:babyId	Unlink a baby from a parent.

9.3 Babies — /babies

Method	Path	Purpose
GET	/	List babies.
GET	/:id	One baby.
GET	/:id/recordings	Recordings for a baby.
POST	/	Create a baby.
PATCH	/:id	Edit a baby.
PATCH	/:id/discharge	Mark discharged.
PATCH	/:id/readmit	Re-admit.
PATCH	/:id/reassign-room	Move to another room.
DELETE	/:id	Delete a baby (cascade).

9.4 Rooms — /rooms

Method	Path	Purpose
GET	/	List rooms.
GET	/available	Rooms with free capacity.
GET	/:id/babies	Babies in a room.
POST	/	Create a room.
PATCH	/:id	Edit a room.
PATCH	/:id/deactivate	Deactivate.
PATCH	/:id/reactivate	Reactivate.
DELETE	/:id	Delete a room (blocked if occupied).

9.5 Recordings — /recordings

Method	Path	Purpose
POST	/	Create/upload a recording (S3 + metadata).
PATCH	/:id/review	Approve/reject/schedule (nurse).

Method	Path	Purpose
POST	<code>/:id/play</code>	Play now on the assigned device.
POST	<code>/:id/stop</code>	Stop playback.
GET	<code>/:id/history</code>	Status-change history.

9.6 Devices — `/devices`

Method	Path	Purpose
GET	<code>/</code>	List speakers (online/offline, assignment).
POST	<code>/</code>	Register a device manually.
POST	<code>/provision</code>	Self-provisioning endpoint (called by the Lambda; needs <code>x-provision-secret</code>).
PATCH	<code>/:id</code>	Edit a device.
PATCH	<code>/:id/assign-baby</code>	Assign/unassign a device to a baby.
DELETE	<code>/:id</code>	Delete a device.

10 Database schema

MySQL. Primary keys are CHAR(36) UUIDs unless noted. Full DDL: `database/schema.sql`.

Table	Purpose & key columns
<code>users</code>	Admins, nurses, parents. <code>role</code> , <code>is_active</code> , <code>invite</code> + <code>reset</code> token fields, <code>password_hash</code> .
<code>rooms</code>	NICU rooms. <code>room_number</code> , <code>capacity</code> , <code>is_active</code> .
<code>babies</code>	Patients. <code>record_number</code> , <code>room_id</code> , <code>status</code> (active/discharged), <code>admission</code> / <code>discharge</code> dates.
<code>parent_baby</code>	Links parents to babies with <code>relationship</code> (primary/secondary).
<code>recordings</code>	Voice messages. <code>baby_id</code> , <code>parent_id</code> , <code>duration_seconds</code> , <code>status</code> (pending_review → scheduled/played/rejected/cancelled), optional AI fields.
<code>recording_status_history</code>	Arbitrary trail of every status change.
<code>schedules</code>	When a recording should play. <code>scheduled_time</code> , <code>trigger_type</code> , <code>status</code> (pending/triggered/cancelled).
<code>devices</code>	Speakers. <code>device_code</code> (PI-#####), <code>serial_number</code> , <code>baby_id</code> , <code>is_online</code> , <code>last_seen_at</code> .
<code>playback_log</code>	Every actual playback: recording, device, who triggered, duration played, source.
<code>schema_migrations</code>	Which migration files have been applied.

11 Frontend deep dive

React 19 SPA built with Vite. `src/api/client.js` is an Axios instance whose base URL comes from `VITE_API_URL`; it attaches the JWT from `AuthContext`. Routing (React Router 7) is in

`src/App.jsx`: public routes (`/login`, `/signup`, `/forgot-password`, `/reset-password`) and role-guarded routes (`/admin/*`, `/nurse/*`, `/parent/*`) via `ProtectedRoute`.

Design system. A single `src/theme.js` plus shared UI primitives in `src/components/ui` (Button, Card, Field, Modal, etc.).

Pages by role. `pages/admin` (with tabs/ for Nurses, Parents, Babies, Rooms, Devices), `pages/nurse`, `pages/parent`.

12 Infrastructure & AWS

12.1 Our server vs. managed services

“The cloud” is just other people’s computers. AWS rents us two *kinds* of things, and the difference matters:

A server we run (EC2)

One rented Linux computer that is *ours to manage*. We install and babysit everything on it: the web server (nginx), our Node app, the TLS certificate. If something on it breaks, we fix it.

Managed services (RDS, S3, IoT Core, SES, Lambda, Route 53)

Pre-built services AWS runs *for* us.

So the mental split is:

ON our EC2 box (we maintain)	Separate AWS services (AWS maintains)
nginx (web server / reverse proxy)	RDS — the MySQL database
our Node.js API (via PM2)	S3 — audio file storage
the built React files nginx serves	IoT Core — the MQTT message broker
the TLS certificate (Certbot)	SES — outbound email
the <code>server/.env</code> secrets	Lambda — the provisioning hook
	Route 53 — DNS for the domain

Why it’s split this way: things that are hard to run correctly and must never lose data (a database, file storage, a message broker) are worth letting AWS manage. The glue that is specific to *us* (nginx routing, our app) lives on our own box where we have full control.

12.2 Networking basics (so the rest makes sense)

IP address

A computer’s numeric address on the internet, e.g. `3.208.245.69`. Our EC2 box has one.

Elastic IP

AWS’s name for a *fixed* public IP you attach to your EC2 box so it doesn’t change when the box restarts. Without it, a reboot could change the address and break the domain.

Domain name

A human name (`voice2baby.com`) that points to an IP, so people don’t memorize numbers.

DNS Translates a domain name into an IP.

Route 53

AWS's DNS service. We use it to say “voice2baby.com and www.voice2baby.com → 3.208.245.69” (these are called **A records**).

Port A numbered door on the box. Web traffic uses 80 (HTTP) and 443 (HTTPS); our Node app listens on 3000 (not exposed to the world ; only nginx talks to it).

Firewall / Security Group

AWS's rules for which ports the box accepts connections on. Ours allows 22 (SSH, for us to log in), 80, and 443. Everything else is blocked.

SSH “Secure Shell” — how we log into the server's command line remotely:

```
ssh -i <private-key-location> ec2-user@3.208.245.69.
```

12.3 EC2: the server itself

EC2 (“Elastic Compute Cloud”) is the rented Linux computer. Think of it as a PC in a data center that never turns off. We SSH in to operate it. It runs Ubuntu/ Amazon Linux, and on it we installed nginx, Node.js, and PM2.

When you touch it: to deploy manually, read logs, edit nginx, or renew certs; otherwise CI/CD does the routine work.

12.4 nginx: the web server and “reverse proxy”

What it is: nginx is a program that listens for web requests from the internet. It's the first thing that answers when someone opens <https://voice2baby.com>.

Its two jobs for us:

1. **Serve the website:** hand the browser the built React files (plain HTML/CSS/JS) for any normal URL.
2. **Reverse proxy the API:** for any URL starting with /api, don't serve a file; instead forward the request to our Node app on port 3000 and pass the answer back. A “reverse proxy” is simply a front-desk that forwards certain requests to a program behind it.

Its config file: /etc/nginx/conf.d/remote-reading.conf. It has the `server_name` (which domains it answers to), the TLS certificate paths, and the two rules above (`location /` → files, `location /api` → proxy to :3000).

When you touch it: adding/changing a domain, or changing routing. After editing, always:

```
sudo nginx -t                # check the config is valid FIRST
sudo systemctl reload nginx   # apply it with zero downtime
```

12.5 HTTPS, TLS, and certificates (the padlock)

TLS The encryption under HTTPS. It scrambles traffic so nobody between the browser and server can read passwords or audio.

Certificate

A file that (a) proves `voice2baby.com` really is us, and (b) carries the key used to set up encryption. Browsers refuse HTTPS without a valid one (that’s the “Not secure” warning).

Certificate Authority (CA)

A trusted issuer that signs certificates. **Let’s Encrypt** is a free, automated CA — that’s who issues ours.

Certbot

The tool on our EC2 box that requests the certificate from Let’s Encrypt, proves we own the domain, installs the cert into nginx, and *auto-renews* it (certs expire every 90 days).

The **certificate files** live at `/etc/letsencrypt/live/voice2baby.com/` — `fullchain.pem` (the certificate) and `privkey.pem` (the private key; secret). nginx’s config points at these.

When you run Certbot: only when adding a *new*

12.6 PM2 — keeping the Node app alive

If you just run `node index.js` and close your terminal, the app stops. Worse, if it crashes, nobody restarts it. **PM2** is a “process manager” that runs the Node app in the background, restarts it automatically if it crashes, and starts it again after a server reboot. Our app is registered under the name `remote-reading-server`. Everyday commands:

```
pm2 status                # is it running?
pm2 logs remote-reading-server # live logs (debugging)
pm2 restart remote-reading-server # after a manual code change
```

12.7 The managed AWS services (what, why, what they need, when)**RDS — the database (managed MySQL)**

A MySQL database server that AWS runs, patches, and backs up for us, so we never administer a database host ourselves.

It has a fixed *endpoint* (a hostname like `voice2baby-db.xxxxx.us-east-1.rds.amazonaws.com`), listens on port 3306, and stores our whole schema (all the tables in Section 10). For security it lives in a *security group* that only accepts connections from the EC2 instance — it is **not** reachable from the public internet, so nobody can hit the database directly. *Needs (in `server/.env`):* `DB_HOST` (the endpoint), `DB_PORT`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`.

S3 — audio file storage

An *object store* (buckets of files). Databases are poor at large binary blobs, so every recording’s audio lives in S3 while the database row only keeps its *key* — the object’s path in the bucket, e.g. `recordings/<uuid>.webm`. The bucket is private. To move a file without making it public or streaming the bytes through our server, S3 issues a **presigned URL**: a temporary, cryptographically signed link that allows *one* operation on *one* object for a few minutes. On upload the browser gets a signed PUT; on playback the Pi gets a signed GET. *Needs:* `AWS_S3_BUCKET`, AWS credentials, and the region. *When:* automatically on every upload and playback.

IoT Core — the MQTT message broker

The service the Pis and the backend meet at to exchange messages over **MQTT**, a lightweight publish/subscribe protocol built for intermittently-connected devices behind firewalls. The server

can't call a Pi directly (Pis have no public address), so *everyone connects out* to IoT Core and communicates through *topics*. The backend publishes a command to `devices/PI-00007/command`; that Pi is subscribed and reacts; the Pi publishes results to `devices/PI-00007/status`, which the backend's subscriber is listening on. Each device's IoT *policy* scopes it to only its own `devices/<its-code>/*` topics, so one speaker can never read or control another. *Auth*: devices use X.509 **certificates**; the backend uses **IAM** (SigV4). *Needs*: the IoT endpoint (`IOT_ENDPOINT` / `AWS_IOT_ENDPOINT`). Devices connect on port 8883, or 443 on locked-down networks.

SES — outbound email

A managed email-sending service used for invite links, password resets, and playback notifications.

Lambda — a “serverless” function

A small piece of code AWS runs *on demand* with no server for us to manage — it wakes up, runs, and stops. We have exactly one, `nicu-preprovision-hook`, and it exists solely for device registration. During a new Pi's provisioning handshake, *AWS IoT Core itself* invokes this Lambda — **the Pi never calls it directly**. The Lambda receives the Pi's hardware serial, calls our backend's `POST /devices/provision` (proving itself with the shared `DEVICE_PROVISION_SECRET`), and returns the allocated `PI-####` so AWS names the new device Thing with it. *Needs* (*Lambda env vars*): `PROVISION_URL` + `PROVISION_SECRET`. *When*: only the first time each Pi registers. Full flow in Section 13.

Route 53 — DNS

Maps `voice2baby.com` to the EC2 Elastic IP (the A records from Section 12).

Accessing RDS from EC2. Because RDS only accepts connections from the EC2 box, you query it *from* EC2 — not from your laptop. SSH into the server, then use the MySQL client with the same credentials that are in `server/.env`:

```
ssh ec2-user@3.208.245.69
# install the MySQL client once, if it's not there:
sudo dnf install -y mariadb105          # (Amazon Linux 2023)
# connect using the values from server/.env:
mysql -h <DB_HOST> -P 3306 -u <DB_USER> -p <DB_NAME>
# it prompts for DB_PASSWORD, then you get a SQL prompt, e.g.:
#   SHOW TABLES;
#   SELECT device_code, is_online FROM devices;
```

Connecting from your laptop will simply time out — that is by design: the database's security group blocks everything except the EC2 instance.

12.8 Credentials: which secret lives where

A frequent source of confusion — *different* parts authenticate *differently*:

- **The backend** → **AWS services** uses **IAM** credentials (an AWS access key/role). IAM = “Identity and Access Management”, AWS's system of who-can-do-what. In production the EC2 box's role grants this; locally you set keys in `.env`.
- **Each Pi** → **IoT Core** uses a unique **X.509 certificate** minted during provisioning — not a password. Certificates are the right fit for physical devices.
- **Browsers** → **backend** use a **JWT** from login.

- **App secrets** (DB password, JWT secret, provisioning secret) live in `server/.env` on the box — never in the code or git.

12.9 At a glance

Service	Kind	Role in Voice2Baby
EC2	our server	Runs nginx + Node (PM2). Static Elastic IP.
nginx	on EC2	Serves the React build; reverse-proxies <code>/api→:3000</code> ; terminates TLS.
Certbot	on EC2	Gets + auto-renews the Let's Encrypt TLS certificate.
PM2	on EC2	Keeps the Node app running / restarts it.
Route 53	managed	DNS: domain → Elastic IP.
RDS (MySQL)	managed	All application data.
S3	managed	Audio files; presigned URLs for up/download.
IoT Core	managed	MQTT broker for play/stop + device status.
SES	managed	Email (may be in sandbox until production access).
Lambda	managed	<code>nicu-preprovision-hook</code> allocates a device code.

13 The Raspberry Pi speaker

13.1 What runs on the Pi

- `pi_subscriber.py` — connects to AWS IoT over MQTT, listens for play/stop commands, downloads audio via presigned URL, and plays it with `ffplay`. Publishes status + a heartbeat.
- `provision_agent.py` — on first boot, self-registers the device.
- `apply-wifi.sh` — configures WiFi from `wifi.txt`.
- systemd units — `nicu-firstboot` (once), `pi-provision`, `pi-subscriber`, and `nicu-wifi` (every boot).

13.2 Zero-touch provisioning: how a blank Pi becomes a known device

The puzzle: every speaker is flashed from the *same* image, so how does a brand-new Pi end up with its own identity (`PI-####`) and its own credentials, with nobody typing anything? The answer is **AWS IoT Fleet Provisioning**. Key point up front: during this whole dance the Pi **only ever speaks MQTT to AWS IoT Core** — it never contacts the Lambda or our backend directly. AWS does that part server-side, on the Pi's behalf.

Baked into every image is one shared **claim certificate** — a deliberately low-privilege “visitor pass” whose only power is to *start* the provisioning handshake (its policy can't read or publish device data). On first boot, `provision_agent.py` runs:

1. It connects to **IoT Core** over MQTT using the shared claim cert.
2. It calls **CreateKeysAndCertificate** — AWS mints a brand-new, unique X.509 certificate + private key for *this* board.

3. It calls **RegisterThing** with our provisioning *template* and the Pi's hardware serial. *This* is the moment AWS IoT Core itself invokes our pre-provisioning **Lambda**, which calls `POST /devices/provision` on the backend and returns the next `PI-####`. AWS then creates the device “Thing” with that name, activates the new certificate, and attaches the per-device policy.
4. The Pi saves its new certificate + code to disk (`device_config.py`) and drops the claim session.

From then on, `pi_subscriber.py` reconnects as its *real* identity (`PI-####`) using the per-device certificate, and the speaker appears in the Speakers tab as Online/unassigned — an admin just clicks *Assign to baby*. Because the backend keys on the hardware serial, re-flashing the same board returns the *same* `PI-####` instead of creating a duplicate.

The one-time AWS setup that creates the claim cert, template, policies, and Lambda is

`scripts/aws/setup-fleet-provisioning.sh` (see `scripts/aws/README.md`). You run it once for the whole fleet.

13.3 Configurable WiFi (`wifi.txt`)

`apply-wifi.sh` reads `wifi.txt` from the boot partition and supports three network types via `security=`:

- `wpa-psk` — home/clinic (`ssid + password`).
- `wpa-eap` — university/enterprise, e.g. eduroam (`ssid + identity + password`; optional `ca_cert + domain_suffix_match`).
- `open` — no password.

It runs on *every* boot (service `nicu-wifi`) but only re-applies when `wifi.txt` changed (checksum-guarded), so the network can be changed in the field by editing the SD card — no reflash.

13.4 The image: what's inside it and how it's modifiable

The speaker image is a **thin image**: rather than build an OS from scratch, we start from the *official* Raspberry Pi OS Lite and only *inject* our own files. What our build (`build-thin-image.sh`) adds or changes:

- `/opt/nicu/` — the first-boot *payload*: `firstboot.sh`, `provision_agent.py`, `pi_subscriber.py`, `provisioning_config.py`, the systemd unit files, and `/opt/nicu/certs/` — the shared **claim certificate** (`claim/claim.cert.pem + claim/claim.private.key`) and `AmazonRootCA1.pem`. (Full cert flow in “Certificates & identity” below.)
- `/usr/local/sbin/nicu-apply-wifi.sh` — the WiFi setup script, kept on a permanent path so it survives first-boot cleanup.
- Enabled systemd services: `nicu-firstboot` (runs once) and `nicu-wifi` (runs every boot).
- On the FAT **boot partition** (the drive a doctor sees as `bootfs`): the editable `wifi.txt` and `login.txt`, a pre-created default user, and SSH enabled.

Two different senses of “modifiable”:

- A *doctor* edits only the plain-text `wifi.txt` (their network), and optionally `login.txt` — on any computer, no rebuild. These live on the FAT boot partition, which Windows and macOS can read.

- *A developer* who changes how the speaker behaves edits files under `pi/` and **rebuilds** the image, then re-hosts it. Crucially, already-deployed Pis keep the version they were flashed with; only newly-flashed cards pick up the change (there is no auto-update).

Building it — two ways (full steps in `BUILD_THIN_IMAGE.md`):

- **Mac + Docker (no extra hardware):** `./pi/build-image-docker.sh` builds inside a privileged Linux container (macOS can't mount the Linux partition itself) and writes `~/v2b-build/voice2baby-s`
- **On a spare Raspberry Pi:** run `pi/build-thin-image.sh` on a Pi acting as a Linux build machine.

Doctors then flash the resulting image, set `wifi.txt`, and power on — see `DOCTOR_SETUP.md`.

13.5 Certificates & identity: which files, and how they're obtained

This has the most moving parts, so here it is file by file. A Pi carries *two* kinds of credential: a **shared claim certificate** (identical on every board, used only to register itself once) and its own **per-device certificate** (unique, created at first boot, used for all normal operation).

Baked into the image (identical on every Pi).

- `/opt/nicu/certs/claim/claim.cert.pem + claim.private.key` — the shared claim certificate and its key. Deliberately low-privilege: its IoT policy can *only* drive the provisioning handshake.
- `/opt/nicu/certs/AmazonRootCA1.pem` — Amazon's root CA, which the Pi uses to verify it is really talking to AWS (server authentication).
- `/opt/nicu/provisioning_config.py` — non-secret coordinates shared by every Pi: the `IOT_ENDPOINT`, the `PROVISIONING_TEMPLATE` name, and the on-disk cert paths.

These are copied from `scripts/certs/claim/` on the build machine by `build-thin-image.sh`. The claim key is secret — that folder is gitignored and must never be committed.

First boot moves them into the user's home. `firstboot.sh` copies the payload into the primary user's home so the user services can read it: `~/certs/AmazonRootCA1.pem`, `~/certs/claim/claim.cert.pem + claim.private.key`, and the three Python files (`provisioning_config.py`, `provision_agent.py`, `pi_subscriber.py`). After first boot succeeds, `/opt/nicu` is deleted; the copies under `~/certs` remain.

How the per-device certificate is obtained — `provision_agent.py`. `provision_agent.py` is the file that obtains the real certificate. It:

1. reads the claim-cert paths from `provisioning_config.py` and connects to `IOT_ENDPOINT` over MQTT using `claim.cert.pem + claim.private.key` (verifying AWS with `AmazonRootCA1.pem`);
2. calls AWS IoT **CreateKeysAndCertificate** — AWS generates and returns a brand-new certificate + private key *for this board*;
3. calls **RegisterThing** with the template and the Pi's hardware serial — this is what triggers the Lambda to allocate the `PI-#####`;
4. its `save_identity()` writes the new credential to disk as `~/certs/<PI-#####>_certificate.crt` and `<PI-#####>_private.key`, and writes `~/device_config.py` containing one line: `DEVICE_CODE = "PI-#####"`.

It is idempotent: if a matching `device_config.py` + cert already exist it exits immediately, so re-running never re-registers.

Normal operation uses only the per-device cert — `pi_subscriber.py`. On every boot after provisioning, `pi_subscriber.py` runs from `device_config` import `DEVICE_CODE`, then connects to AWS IoT as its real identity using `~/certs/<DEVICE_CODE>_certificate.crt` + `<DEVICE_CODE>_private.key` (again verifying AWS with `AmazonRootCA1.pem`). The claim cert is never used again. Its topics are `devices/<DEVICE_CODE>/play`, `/stop`, and `/status`.

In one sentence. The **claim** cert (`claim.cert.pem`) is the file used *to obtain* the real certificate; `provision_agent.py` is the program that obtains it; the result is the **per-device** cert (`<PI-#####>_certificate.crt`) that `pi_subscriber.py` then uses for good — and `AmazonRootCA1.pem` verifies AWS in both cases.

14 Deployment & operations

14.1 CI/CD

Pushing to main triggers `.github/workflows/deploy.yml`: it SSHes to EC2, pulls, `npm install`, runs migrations (automatically on server start), restarts PM2 (`voice2baby/remote-reading-server`), and rebuilds + copies the client build that nginx serves.

14.2 Manual operations on EC2

```
pm2 status                # process state
pm2 restart remote-reading-server
pm2 logs remote-reading-server # tail logs
sudo nginx -t && sudo systemctl reload nginx
cd server && npm run migrate    # apply migrations without a restart
```

14.3 Domain & TLS

Live domain `voice2baby.com` on Route 53; TLS via Certbot (auto-renews). The full cutover runbook is `DOMAIN_SETUP.md`.